Technical University of Crete
School of Electrical and Computer Engineering
Course: Randomized Algorithms
Exercise 1 (100/1000)
Report Delivery Date: 22/10/2024
Instructor: Athanasios P. Liavas

Paraskevopoulou Despoina
2021030135

Coupon Collector

In this exercise we simulated the Coupon Collector problem. More specifically, we assumed that there exist n different types of coupons and we have to select coupons until we collected each coupon from each type. In our simulation in Matlab, we have a vector of coupons size $1 \times n$ filled with zeros and every time we choose uniforlmy (using randi) the coupon we are collecting. The coupon takes the value 1 and so we continue in order to fill the vector with ones. We repeated the experiment for K times and we plotted the number of required selections for each experiment.

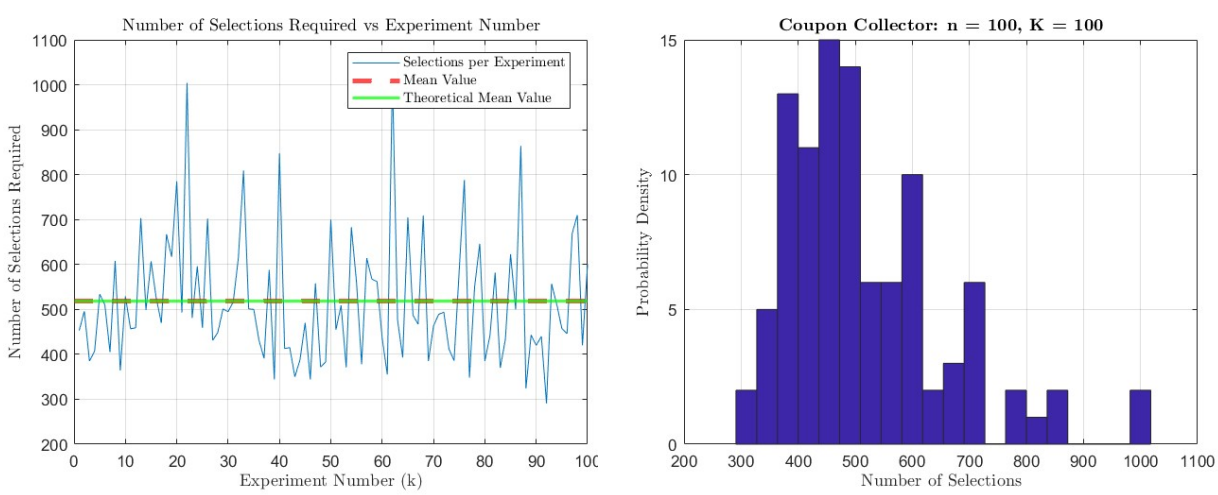
Also, we computed and plotted the mean value of the selections, as well as, the histogram of the number of the required selections for the various experiments. For a better visualization of the results, we also computed the theoretical mean value of the selections, the expected number of random coupons required to obtain all n coupons:

$$\mathbb{E}[X] = n \sum_{i=1}^{n} \frac{1}{i}$$

The summation $\sum_{i=1}^{n} \frac{1}{i}$ is known as the harmonic number $H(n)$ and,

$$H(n) = \ln n + \Theta(1).$$

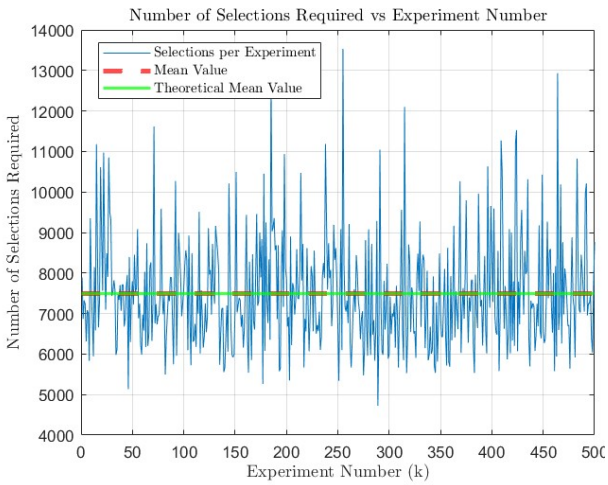
so, the theoretical mean value of the selections is $n \ln n$, which means that we have complexity of $O(n \log n)$.
We derived the following results:



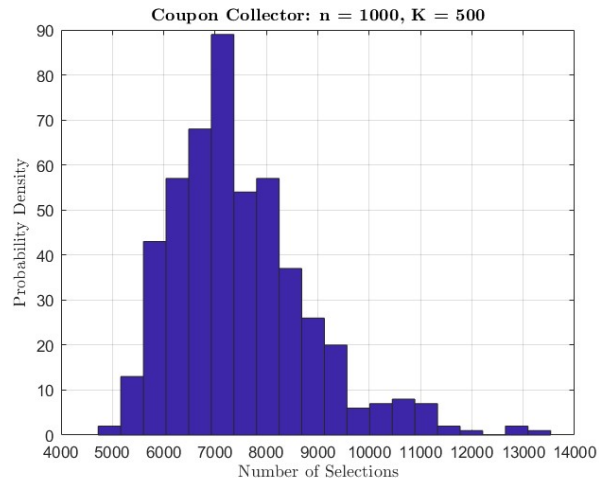
(a) Number of Selections Required vs Experiment Number

(b) Histogram of the selections for the various experiments

Figure 1: Comparison of selection process and its histogram for n=100, K=100



(a) Number of Selections Required vs Experiment Number



(b) Histogram of the selections for the various experiments

Figure 2: Comparison of selection process and its histogram for n=1000, K=500

From the results, we observe that the theoretical mean value is almost the same with the experimental. Furthermore, we see that the histograms tend to follow a binomial distribution, especially for a high K.

The code for the implementation is:

```
function N = coupon_collector(n,K)
    N = zeros(1,K);
    for k=1:K
        collectedCoupons = zeros(1, n);
        total = 0;
        numCollected=0;

        while numCollected < n
            coupon = randi(n);
            total = total+1;

            if (collectedCoupons(coupon) == 0)
                collectedCoupons(coupon) = 1;
                numCollected = numCollected+1;
            end
        end

        N(k) = total;
    end
end
```

Quick Sort

Moving on, we are going to implement the QuickSort algorithm for n elements. First, we implemented the Partition algorithm and then we moved on to the QuickSort algorithm, using Partition.

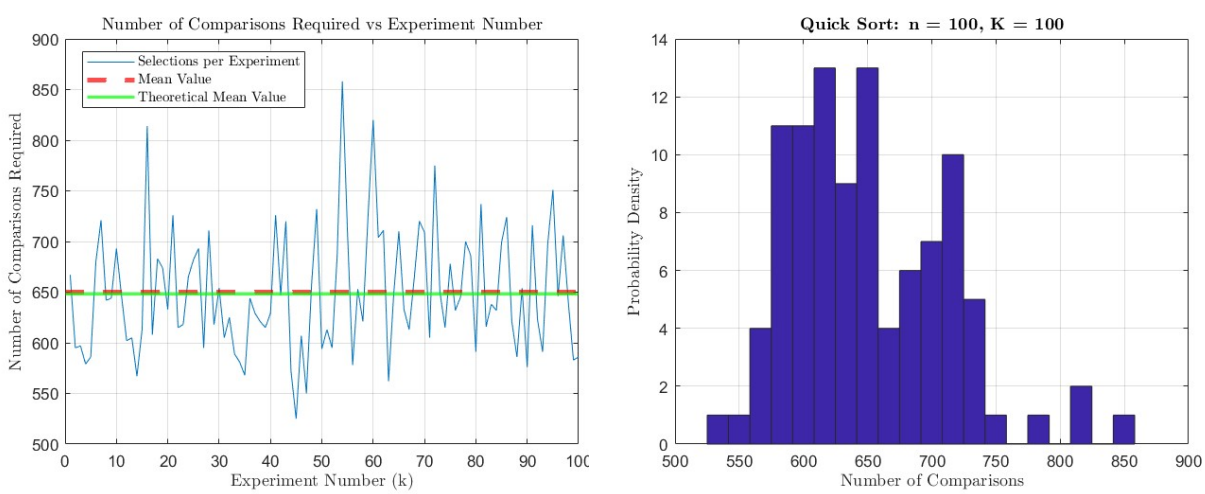
In our Partition algorithm, we choose a pivot randomly and we divide the array in two parts, the left one, where all elements are smaller than the pivot and the right one where all elements are larger than the pivot. Then in out Quick Sort algorithm, we call Partition for our initial array and then with recursive calls of the Quick Sort, we sort our array.

In this exercise, as before, we computed the theoretical mean value of comparisons which is given by

$$\mathbb{E}[X] = 2(n+1)H(n) - 4n$$

where $H(n) = \sum_{i=1}^{n} \frac{1}{i} = \ln n + \Theta(1)$. So, we understand, again, that we have a complexity of $O(n \log n)$.

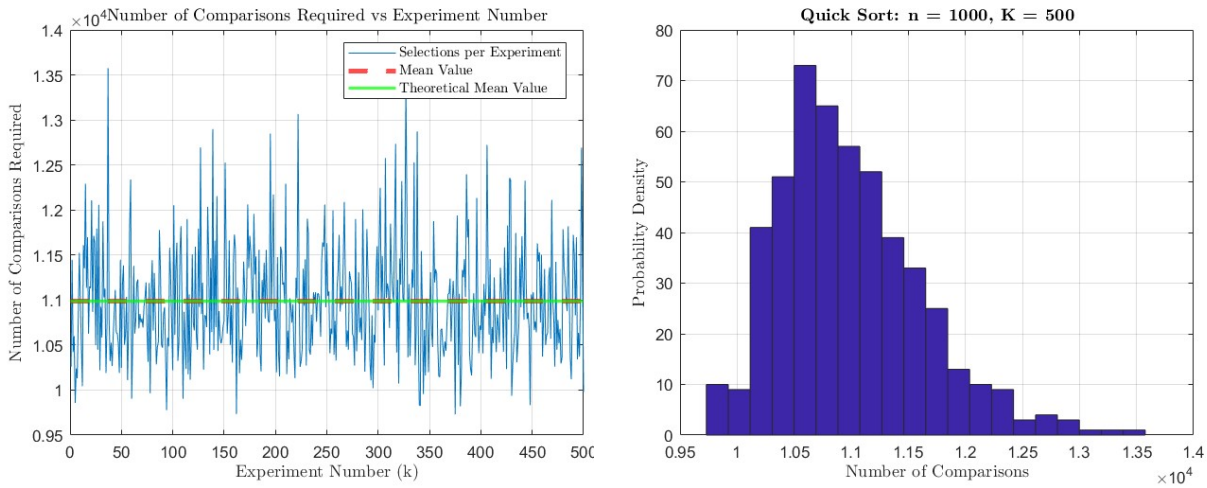
We repeated the experiment for K=100 and K=500 and we got the following results. We computed the number of comparisons for the number of algorithm runs and also the average comparisons for each run (for each K).



(a) Number of Comparisons Required vs Experiment Number

(b) Histogram of the comparisons for the various experiments

Figure 3: Comparisons and Histogram for n=100 and K = 100



(a) Number of Comparisons Required vs Experiment Number

(b) Histogram of the comparisons for the various experiments

Figure 4: Comparisons and Histogram for n=1000 and K = 500

From the plots, we noticed that the experimental and the theoretical mean values are close, and we also checked that the number of comparisons is close to $n \log n$.
The code for the Partition Algorithm is:

```
function [pivot,comps,left,right] = partition(S,high,low)
    n = length(S);
    pivot_i = randi([low, high]); pivot = S(pivot_i);
    comps=0; right=[]; left = [];
    for j = 1:length(S)
        if (S(j)< pivot)
            comps = comps + 1;
            left = [left,S(j)];
        end
        if(S(j)>pivot)
            comps = comps + 1;
            right = [right,S(j)];
        end
    end
end
```

The code for the Quick Sort Algorithm is:

```
function [total,sorted_final] = quick_sort(sorted,high,low)
    total = 0;
    if low >= high
        sorted_final = sorted;
        total = 0;
        return;
    end
    if low<high
        [pivot,comp,sorted_final1,sorted_final2] = partition(
           sorted,high,low);
        [total1,sorted_final1] = quick_sort(sorted_final1,
           length(sorted_final1),1);
        [total2,sorted_final2] = quick_sort(sorted_final2,
           length(sorted_final2),1);
        total = total1 + total2 + comp;
        sorted_final = [sorted_final1, pivot,sorted_final2];
    end

end
```

Randomized Median

For the last part of the exercise, we studied and implemented the Randomized Median algorithm, for input n. For the implementation we followed the indications given in theory. For the sorting of the arrays, we used the quick sort algorithm that we made in the previous exercise.

This time, the expected number of comparisons is given by

$$\mathbb{E}[X] = 2n + o(n)$$

This results from the following analysis of the algorithm.

- Sampling the multiset R of $n^{\frac{3}{4}}$ elements of S with replacement and sorting them. This takes $O(n^{\frac{3}{4}} logn^{\frac{3}{4}}) = o(n)$ comparisons.
- Creating the set C takes O(n) comparisons and creating the interval of $l_{(d)}$ and $l_{(u)}$ takes 2n comparisons exactly by comparing every element with both $l_{(d)}$ and $l_{(u)}$.
- The checks of failure in the algorithm take O(1) comparisons.

Thus,the total expected number of comparisons is given by $T(n) = 2n + o(n)$.
We computed the number of comparisons that are required and we plotted the results.

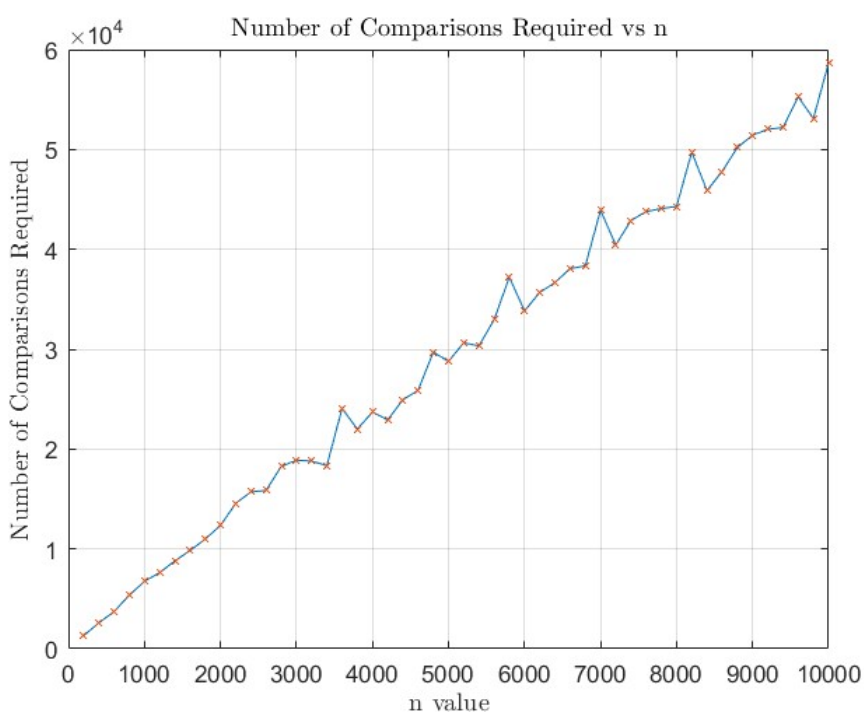


Figure 5: Number of Comparisons vs n

We used our Quick Sort implementation for the Randomized Median algorithm. From the plot of the comparisons with respect to n it is confirmed that the algorithm has a complexity of $2n + o(n)$. We see that the graph is not entirely linear, and that is due to the randomization of the Median algorithm and, also, the Quick Sort algorithm that it was used. If instead of the randomized Quick Sort algorithm we used a deterministic algorithm, the graph would be smoother.
The code for the Randomized Median Algorithm:

```
function [median,comp,comp_th] = findMedian(S)
    n = length(S);
    sample = ceil(n^(3/4));
    R = randsample(n,sample);
    comp_th = 0;
    comp = 0;

    [comp1,sorted_R] = quick_sort(R,length(R),1);
    comp = comp + comp1;
    comp_th = comp_th + length(R)*log(length(R));

    m = (n^(3/4))/2;
    di = floor(m - sqrt(n));
    ui = ceil(m + sqrt(n));

    d = sorted_R(di);
    u = sorted_R(ui);

    C = []; Cd = []; Cu = [];
    for i = 1:n
        comp = comp + 2;
        comp_th = comp_th  + 2;
        if(S(i) >= d && S(i) <= u)
            C = [C; S(i)];
        elseif(S(i) < d)
            Cd = [Cd; S(i)];
        else
            Cu = [Cu; S(i)];
        end
    end
```

```

    ld = length(Cd);
    lu = length(Cu);

    comp = comp+2;
    comp_th = comp_th + 2;
    if (ld>n/2 || lu>n/2)
        median = -1;
        return;
    end

    comp = comp+1;
    comp_th = comp_th + 1;
    if (length(C) <= 4*n^(3/4))
        [comp2,sorted_C] = quick_sort(C,length(C),1);
        comp_th = comp_th + length(C)*log(length(C));

        comp = comp  + comp2;

    else
        median = -1;
        return;
    end

    median_i = floor(n/2) - ld +1;
    median = sorted_C(median_i);

end
```